

Sentinel AIOps:

AI-Powered Incident Intelligence

An agentic AIOps platform that correlates metrics, logs, and distributed traces, then drives a 10-node LangGraph inference pipeline to detect, classify, forecast, and manage infrastructure incidents — all in real time, without human triage.

AI Incident Intelligence

Metrics + Logs + Traces

LangGraph StateGraph

LightGBM Classifier

Forecasting + TTF

Weibull + Kaplan-Meier

Human-in-the-Loop Gate

FastAPI + React + Vite

QueryLangGraph Chatbot

SSE Real-Time Push

13

FAILURE
MODES

10

PIPELINE
NODES

37

ML
FEATURES

20+

API
ENDPOINTS

11

DB TABLES

PROJECT

Sentinel AIOps

STACK

• Python • FastAPI • LangGraph • LightGBM • SQLite •
React/Vite

COMPILED

August 2026

What This Document Covers

Sentinel AIOps — from the problem it solves to every algorithm, formula, API endpoint, and database table it uses.

#	SECTION	WHAT YOU WILL UNDERSTAND AFTER READING IT
01	The Incident Problem	Why traditional monitoring creates alert overload, manual triage, and MTTD blindspots
02	What Sentinel AIOps Is	The complete platform — objectives, operating modes, scope, and impact
03	System Architecture	Four-process architecture, communication map, port assignments, unified launch
04	Telemetry Architecture	Metrics, logs, traces — the three pillars, 13 failure modes, and how telemetry is correlated
05	Feature Engineering	How 37 features are extracted from raw telemetry — 32 metrics + 5 log features + CB encoding
06	ML Classification- LightGBM	Gradient boosting model, Optuna HPO, Drain3 log parsing, auto-training, evaluation
07	LangGraph Inference Pipeline	All 10 nodes, the AIOpsLangState schema, conditional routing, checkpointing, error handling
08	Tumbling Window	10-cycle majority-vote smoother — formula, worked example, margin, flap suppression
09	Forecasting & Time-to-Failure	Mode-specific routing, ARIMA, linear, exponential algorithms — formulas and examples
10	Severity Update & Hysteresis	4-step Impact × Urgency matrix, dwell counter formula, escalation vs de-escalation logic
11	Reliability & Survival Analysis	Kaplan-Meier and 2-Parameter Weibull MLE — formulas, 4 failure groups, survival probability
12	Human-in-the-Loop Gate	Escalation trigger condition, review lifecycle, auto-approval timeout, audit trail
13	QueryLangGraph Chatbot	11-node guarded chatbot pipeline — guardrails, intent routing, retrieval, LLM synthesis
14	API & SSE Architecture	All 20 REST endpoints + SSE broadcaster — request-response flows, subprocess management
15	Frontend Dashboard	React/Vite SPA — 13 components, SSE lifecycle, live feed toggle, chatbot panel
16	Database & Persistence	Two SQLite databases, WAL mode, all 11 table schemas, DbWriter atomic writes
17	Technology Stack & Conclusion	Full technology rationale, outputs, quick reference, 10 golden rules, future scope

ONE-LINE SUMMARY

Sentinel AIOps transforms raw microservice telemetry into a classified, severity-graded, time-to-failure-predicted incident record — autonomously, in under 500 milliseconds per cycle, using a 10-node LangGraph AI pipeline.

The Incident Problem

Modern cloud-native infrastructure generates thousands of telemetry signals per second — but the tools to make sense of them in real time have not kept pace with system complexity.

A production microservice environment is never truly static. Services scale, traffic patterns shift, dependencies fail, and code deploys introduce regressions. When something goes wrong, the on-call engineer is immediately presented with an overwhelming volume of signals: hundreds of Prometheus alerts, dozens of Grafana dashboards, a wall of error logs in Kibana, and dozens of Datadog traces — all simultaneously.

The fundamental challenge is not a shortage of observability data. It is the cognitive burden of correlating that data in real time, under pressure, to identify the **failure mode**, its **root cause**, the correct **severity**, and the projected **time remaining before critical threshold breach**.

Core Production Challenges

CHALLENGE	WHAT IT MEANS	BUSINESS IMPACT
Alert Storm	100+ alerts fire simultaneously for one root cause	Engineer paralysis; true signal buried in noise
Manual Correlation	Engineer must mentally join CPU, memory, DB latency, logs across 5+ dashboards	MTTD of 15–45 minutes for cascading failures
Static Severity	P1/P2/P3 assigned from a single metric threshold	Misclassification → under-reaction or false-alarm fatigue
No TTF Prediction	Teams are notified after a threshold is breached, not before	Reactive posture; damage already done
Siloed Signals	Metrics, logs, and traces analyzed independently	Multi-signal failures (e.g., CASCADING_FAILURE) missed
No Audit Trail	Severity decisions made verbally or in chat	Post-mortem is guesswork; no accountability
Query Barrier	Engineers must write SQL / PromQL to investigate historical incidents	Senior engineer time wasted on data retrieval

BEFORE SENTINEL

- Manual alert correlation from 5+ dashboards
- MTTD: 15–45 minutes for complex failures
- P1/P2 based on a single metric breach
- No time-to-failure visibility
- Severity decisions: unaccountable, verbal

WITH SENTINEL AIOPS

- Automated multi-signal correlation per cycle
- Detection in <10 seconds via LangGraph pipeline
- Compound severity: impact × urgency × forecast
- ARIMA/linear TTF prediction per failure mode
- Full audit trail in SQLite (every decision recorded)

THE CORE INSIGHT

The problem is not that monitoring systems lack data — it is that data correlation, failure classification, and severity judgment require a cognitive load that scales with system complexity. When a senior engineer leaves the on-call rotation, that institutional knowledge leaves with them. Sentinel makes that knowledge executable.

What Sentinel AIOps Is

Sentinel is not a monitoring tool. It is an autonomous AI agent that transforms raw telemetry into classified, severity-graded, time-to-failure-predicted incident records — in real time.

Sentinel AIOps is a multi-layer agentic platform built around a **LangGraph StateGraph** — a directed graph of AI/ML analysis nodes that share a common state object and execute in sequence for every telemetry tick. Each node has exactly one responsibility, and the graph routes data through those responsibilities with conditional error handling.

Three Operating Modes

MODE 01

Historical Pipeline

Processes pre-generated simulator data from `simulator_db.sqlite` for model training, evaluation, and replay.

MODE 02

Live Feed Mode

Streams real-time simulated telemetry from `live_feed_db.sqlite` through the full 10-node inference pipeline.

MODE 03

Chat Query Mode

Enables natural-language queries over the incident database via an 11-node LLM-guarded `QueryLangGraph` agent.

Project Objectives

- 1

Automate failure detection
Across 13 cloud-native failure modes with ≥95% classification accuracy using LightGBM with Optuna HPO.
- 2

Reduce MTTD to <10 seconds
From 15–45 minutes (manual triage) to sub-10-second automated detection per 500ms pipeline cycle.
- 3

Provide TTF prediction
Mode-specific ARIMA/linear/exponential forecasters compute seconds until threshold breach.
- 4

Compound severity scoring
Revised P1–P4 accounts for current impact + forecast urgency + hysteresis stabilization.
- 5

Human-in-the-Loop Gate
Operator approval workflow for large severity jumps (≥2 levels), with full audit trail.
- 6

Natural-language incident queries
Guarded LLM agent pipeline answers questions about past and current incidents without SQL.
- 7

Real-time dashboard via SSE
React frontend subscribes to Server-Sent Events, rendering updated incident state every 500ms.

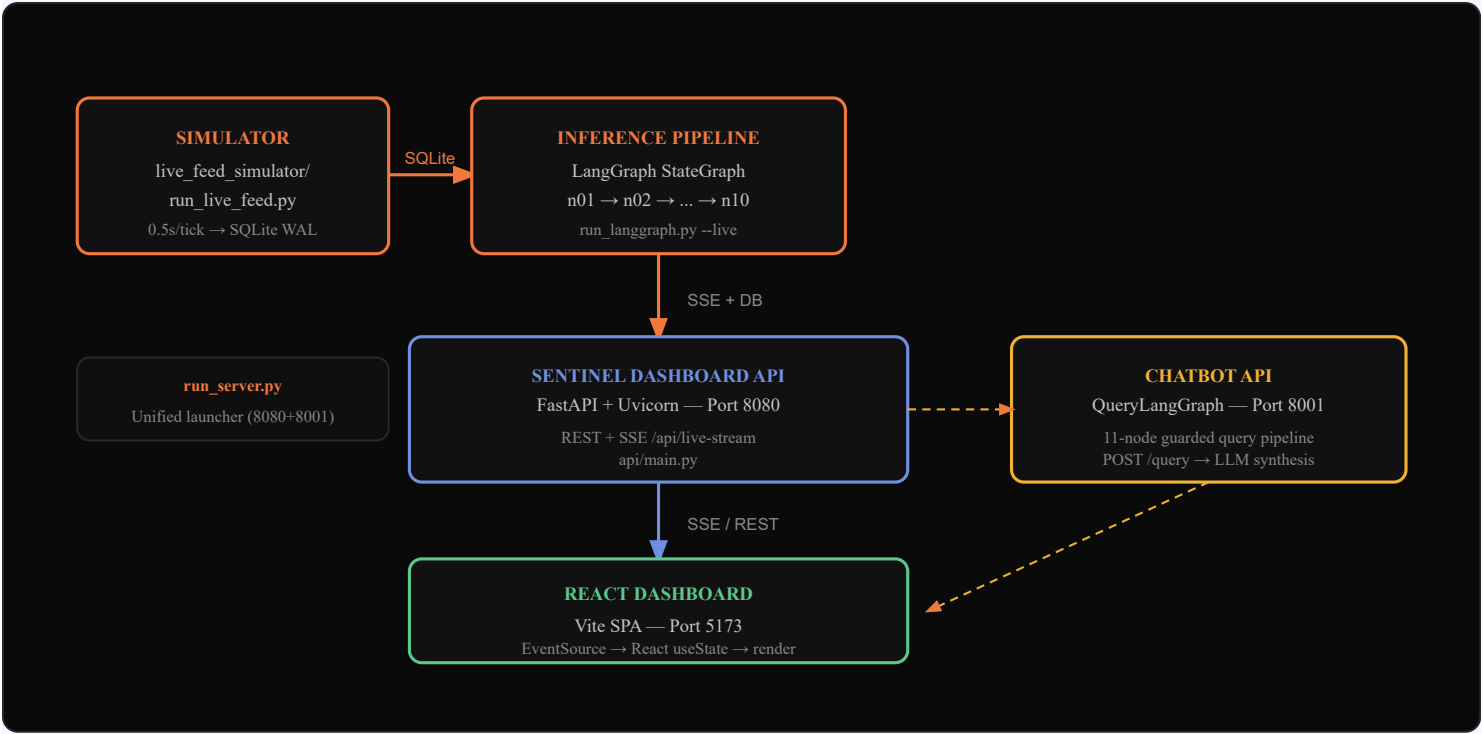
Project Scope

Telemetry simulation for 13 failure modes across 4 microservices · 10-node LangGraph inference pipeline · Full SQLite schema (11 tables) · Kaplan-Meier + Weibull survival analysis · FastAPI REST + SSE (port 8080) · QueryLangGraph chatbot (port 8001) · React/Vite dashboard (port 5173) · Live feed simulator with WAL-mode SQLite · Human Gate with timeout auto-approval.

Production Kubernetes deployment · Real Prometheus/OpenTelemetry integration · Multi-tenant support · Authentication/authorization layer · GPU-accelerated inference · External alerting (PagerDuty/Slack/OpsGenie) · Distributed SQLite (replaced by single-node WAL mode).

System Architecture

Sentinel runs as four independent Python processes communicating through SQLite, REST, and SSE — unified by a single launcher script.



Port & Process Map

PROCESS	PORT	ENTRY POINT	TRIGGERED BY	TYPE
Sentinel Dashboard API	8080	api/main.py	run_server.py (main)	HTTP + SSE
QueryLangGraph Chatbot API	8001	start_chatbot_server.py	run_server.py (subprocess)	HTTP
LangGraph Inference Pipeline	–	run_langgraph.py	POST /api/live/start	BACKGROUND PROCESS
Live Feed Simulator	–	run_live_feed.py	POST /api/live/start	BACKGROUND PROCESS
React Frontend Dev Server	5173	npm run dev	Manual (separate terminal)	DEV SERVER

STARTUP SEQUENCE

Terminal 1: `python backend/run_server.py` — starts Chatbot API (8001) as a background subprocess, then blocks on Uvicorn serving the Dashboard API (8080).

Terminal 2: `cd frontend && npm run dev` — starts the React/Vite dev server on port 5173.

The simulator and LangGraph pipeline start only when the operator clicks **"Start Live Feed"** in the UI, triggering `post /api/live/start`.

Telemetry Architecture

Three telemetry pillars — metrics, logs, and distributed traces — model a realistic four-service microservice environment and are the raw material for everything the AI pipeline reasons over.

PILLAR 01 — METRICS

32 Numeric Signals

CPU, memory, latency percentiles, error rates, GC pauses, DB query times, cache ratios, queue depth, network errors, circuit breaker state — one row per tick per service in the metrics table.

PILLAR 02 — LOGS

Structured Log Events

Log level (INFO/WARN/ERROR/CRITICAL), exception type (RuntimeException, SocketTimeoutException, etc.), and message text — one row per log entry per tick in the logs table.

PILLAR 03 — TRACES

Distributed Spans

Trace ID, span ID, parent span ID, span name, duration_ms, span status (OK/ERROR), and peer service — 2–5 spans per tick modeling request propagation in the traces table.

13 Failure Modes

FAILURE MODE	PRIMARY SIGNAL INJECTION	LOG LEVEL	EXCEPTION
NONE	All metrics at healthy baseline	INFO	—
MEMORY_LEAK	heap_mb grows linearly; gc_pause_p99 increases	WARNING	—
CPU_SATURATION	cpu_utilization ramps to 95%+; p99_latency degrades	WARNING	—
LATENCY_SPIKE	p99_latency spikes; gc_pause_p99 elevated	WARNING	—
ERROR_STORM	error_rate + http_5xx_rate spike	ERROR	RuntimeException
DB_SLOWDOWN	db_p99 + db_connection_wait increase	WARNING	—
CACHE_STAMPEDE	cache_miss_rate spikes; db_p99 secondary increase	WARNING	—
QUEUE_BACKUP	queue_lag + thread_pool_queue saturate	WARNING	—
DEPENDENCY_TIMEOUT	upstream_timeout_rate high; p99_latency degrades	ERROR	SocketTimeoutException
BAD_DEPLOY	error_rate + http_5xx_rate immediate spike	ERROR	NullPointerException
RETRY_STORM	retry_count_per_request + rps both high	WARNING	—
DISK_IO_SATURATION	iops_utilization + disk_read_latency saturate	WARNING	—
CASCADING_FAILURE	Multiple metrics breach P1 simultaneously	CRITICAL	SystemOverloadException

Healthy Baseline Values

Metric	Healthy Baseline	Metric	Healthy Baseline
cpu_utilization	22.0%	p50_latency	90 ms
memory_utilization	25%	p99_latency	130 ms
heap_mb	510 MB	error_rate	1%
gc_pause_ms	15 ms	cache_hit_ratio	95%
db_query_latency	21 ms	network_errors	2/tick

Gaussian noise of $\pm 4\%$ (`NOISE_LEVEL = 0.04`) is applied to all metric values to simulate real-world jitter and prevent the classifier from memorizing deterministic patterns.

Temporal Correlation (n01_collect)

Every telemetry tick is correlated across all three pillars by **(episode_id, timestamp)**. The collect node issues three sequential SQL queries against the same SQLite database:

```
N01_COLLECT.PY - TEMPORAL JOIN

# Step 1: Fetch next metric row by monotonic id
metric = conn.execute("SELECT * FROM metrics WHERE id > ? LIMIT 1", (last_id,))

# Step 2: Fetch all log entries at the same (episode_id, timestamp)
logs    = conn.execute("SELECT log_level, exception_type, log_message
                        FROM logs WHERE episode_id = ? AND timestamp = ?", (ep_id, ts))

# Step 3: Fetch all trace spans at the same (episode_id, timestamp)
spans   = conn.execute("SELECT * FROM traces WHERE episode_id = ? AND timestamp = ?", (ep_id, ts))
```

This produces a perfectly correlated snapshot: metrics, logs, and traces share the same `episode_id` and `timestamp`, guaranteeing that feature engineering receives a coherent multi-signal view of a single point in time.

Feature Engineering

Raw correlated telemetry — a metric row, a set of log entries, and a set of trace spans — is transformed into a 37-dimensional numeric feature vector that LightGBM can reason over in under a millisecond.

Node **n02_feature_engineering** is the boundary between raw observability data and machine-readable signal. It performs no ML — it is pure signal extraction, normalization, and encoding. The 37-feature vector is the sole input to both the preliminary severity engine (rule-based) and the LightGBM classifier (ML-based).

Feature Vector Composition

Metric Features (32)

Direct extraction from `raw_metric` with type coercion to float. The `circuit_breaker_state` field is ordinally encoded.

cpu_utilization	memory_utilization	heap_mb	db_p99	disk_read_latency	disk_write_latency
error_rate	gc_pause_p99	cache_hit_rate	cache_miss_rate	active_connections	network_errors
p50_latency	p95_latency	p99_latency	queue_lag	retry_count	rps
upstream_timeout	cb_state (enc)	http_4xx_rate	http_5xx_rate	iops_utilization	thread_pool_queue
cpu_saturation	db_connection_pool	db_connection_wait	disk_io_wait	rtt_ms	queue_depth
gc_pause_p50	retry_budget_used				

Log Features (5)

Derived from `raw_log` list via aggregation. Drain3 provides template novelty detection.

FEATURE	DERIVATION
log_count	len(raw_log)
log_max_severity	max(level_numeric) — INFO=0, WARN=1, ERROR=2, CRIT=3
log_critical_count	count(level == CRITICAL)
log_has_exception	1 if any log has exception_type, else 0
log_has_novel_template	1 if Drain3 sees unseen template, else 0

Circuit Breaker Encoding

The `circuit_breaker_state` string is ordinally encoded before being passed to LightGBM — the ordinal order matches the failure progression.

```
# Ordinal encoding – must match training data
CB_ENCODE = {
    "closed": 0, # Normal
    "half-open": 1, # Recovering
    "open": 2, # Failing
}
```

DESIGN DECISION

No dimensionality reduction (PCA, t-SNE) is applied before passing features to LightGBM. Gradient boosted trees handle 37 features efficiently without transformation — reducing features would risk losing the specific signal combinations (e.g., **high heap_mb AND high gc_pause_p99**) that distinguish MEMORY_LEAK from LATENCY_SPIKE.

Drain3 — Log Template Mining

Drain3 is a streaming log parser that identifies structured templates from free-text log messages. For example, the message `"WARN heap=847MB gc_p99=312ms -- memory leak"` is parsed into the template `"WARN heap=<*>MB gc_p99=<*>ms -- memory leak"`. When Drain3 sees a template it has never encountered before, it sets `log_has_novel_template = 1`, which is a strong signal for failure mode transitions — particularly BAD_DEPLOY (where a new code deployment introduces entirely new log patterns) and CASCADING_FAILURE.

ML Classification — LightGBM

A LightGBM gradient-boosted tree classifier, tuned by Optuna's Tree-structured Parzen Estimator, identifies the failure mode from the 37-feature vector with sub-millisecond inference latency.

Why LightGBM?

Alternative	Limitation for Sentinel	LightGBM Advantage
XGBoost	Level-wise tree growth slower on wide data	Leaf-wise growth: faster convergence on 37 features
Random Forest	No sequential error correction; weaker on imbalanced data	Boosting corrects systematic errors between trees
LSTM/Transformer	Requires sequential fixed-length input; heavy inference	Single-row inference; no sequence needed per cycle
SVM	Slow training on 120k+ rows; no native probability output	Native predict_proba() gives confidence for TTF gating

Hyperparameter Tuning — Optuna TPE

Optuna's **Tree-structured Parzen Estimator (TPE)** sampler performs a directed Bayesian search over 9 hyperparameters. Each trial trains a LightGBM model and evaluates it on a held-out fold using **macro-averaged F1**, giving equal weight to all 13 failure mode classes regardless of frequency.

Configuration	Value	Rationale
Trials	80	Practical sweet-spot for 9-parameter LightGBM search
CV Folds	5	StratifiedKFold: 20% per fold, stable per-class F1 estimates
Metric	macro F1	Equal weight to all 13 classes; penalizes minority mode neglect
Early Stopping	50 rounds	Stop adding trees if validation F1 hasn't improved in 50 rounds
Random Seed	42	Shared across Optuna, StratifiedKFold, and train/test split
Train/Test Split	80/20	Held-out test set never seen during tuning
Study Persistence	optuna_study.db	SQLite-backed; interrupted tuning runs are resumable

Search Space (9 Parameters)

OPTUNA TRIAL DEFINITION

```
params = {
    "n_estimators":    trial.suggest_int("n_estimators",    100, 1000),
    "learning_rate":   trial.suggest_float("learning_rate", 0.01, 0.3, log=True),
    "max_depth":       trial.suggest_int("max_depth",       3, 10),
    "num_leaves":      trial.suggest_int("num_leaves",      16, 256),
    "min_child_samples": trial.suggest_int("min_child_samples", 5, 100),
    "reg_alpha":       trial.suggest_float("reg_alpha",     1e-8, 10.0, log=True),
    "reg_lambda":      trial.suggest_float("reg_lambda",    1e-8, 10.0, log=True),
    "feature_fraction": trial.suggest_float("feature_fraction", 0.4, 1.0),
    "bagging_fraction": trial.suggest_float("bagging_fraction", 0.4, 1.0),
}
```

Runtime Inference

At inference time, a single-row DataFrame is constructed from the 37-feature dict, ordered by the saved `feature_names.json`, and passed to the loaded model:

N04_CLASSIFY.PY — INFERENCE PATH

```
df = pd.DataFrame([features_dict])[feature_names]    # enforce column order
proba = model.predict_proba(df)[0]                  # shape: (13,)
idx = proba.argmax()
predicted = label_encoder.inverse_transform([idx])[0] # e.g. "MEMORY_LEAK"
confidence = float(proba[idx])                        # e.g. 0.94
```

Graceful Degradation — Auto-Train

If `lgbm_model.pkl` is missing at pipeline startup, the classifier checks row count in `engineered_features.csv`. If `MIN_TRAIN_ROWS = None` (or enough rows exist), it launches `train_classifier.py` as a subprocess, blocks until completion, then loads the freshly trained model. While training, it returns **"PENDING"** with 0.0 confidence so downstream nodes can continue without crashing.

Preliminary Severity — Rule Engine

Before the ML classifier runs, a fast rule-based engine assigns an initial P1–P4 severity from five critical metric thresholds. It takes microseconds and requires no model.

Node **n03_prelim_severity** uses explicit threshold conditions from `config.py`. The engine evaluates checks in descending severity order and returns the first severity level that is triggered. If no threshold is breached, the service is classified as **P4 (healthy baseline)**.

Severity Levels

LEVEL	NAME	RESPONSE SLA	COLOR
P1	CRITICAL	Immediate response required	■ Red
P2	HIGH	Response within 15 minutes	■ Orange
P3	MEDIUM	Response within 1 hour	■ Yellow
P4	LOW	Monitor, no immediate action	■ Green

Threshold Conditions (from config.py)

```
PRELIMINARY SEVERITY — ALL THRESHOLD CONDITIONS

# P1 — CRITICAL: ANY of these conditions → immediate response
P1 if cpu_utilization    > 90    # reason: "cpu=91.2%>90%"
P1 if memory_utilization > 90    # reason: "mem=92.1%>90%"
P1 if p99_latency        > 1000  # reason: "p99=1043ms>1000ms"
P1 if log_critical_count > 5     # reason: "critical_logs=6>5"
P1 if error_rate         > 0.10  # reason: "err=12.4%>10%"

# P2 — HIGH: ANY of these (only if no P1 triggered)
P2 if cpu_utilization    > 80
P2 if memory_utilization > 80
P2 if p99_latency        > 700
P2 if error_rate         > 0.05
P2 if gc_pause_p99       > 200
P2 if log_has_exception  == 1

# P3 — MEDIUM: ANY of these (only if no P1/P2 triggered)
P3 if cpu_utilization    > 60
P3 if p99_latency        > 400
P3 if error_rate         > 0.02
P3 if cache_miss_rate    > 0.50

# P4 — LOW: all metrics within normal range
P4 otherwise # reason: "baseline — all metrics within normal range"
```

IMPORTANT: WHY PRELIMINARY SEVERITY EXISTS

The LightGBM classifier identifies **what kind** of failure is occurring. The preliminary severity engine answers **how bad is it right now** — purely from threshold violations. These are complementary signals: an ERROR_STORM episode might be P1 while a MEMORY_LEAK early in its trajectory is P3. Both assessments flow into the severity update node together.

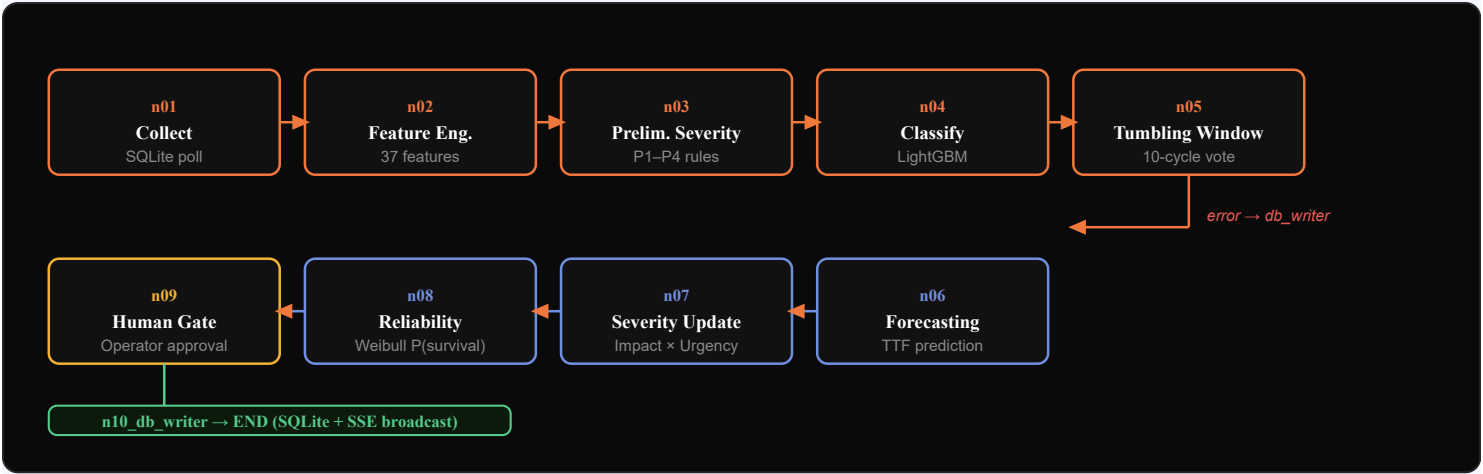
Worked Example

METRIC	MEASURED VALUE	P1 THRESHOLD	TRIGGERED?	REASON STRING
cpu_utilization	91.2%	90%	✓ YES	cpu=91.2%>90%
p99_latency	1,050 ms	1,000 ms	✓ YES	p99=1050ms>1000ms
memory_utilization	78%	90%	✗ NO	—

Result: **P1 — CRITICAL** · Reason string: "cpu=91.2%>90%, p99=1050ms>1000ms"

The LangGraph Inference Pipeline

Ten specialized analysis nodes, assembled into a LangGraph StateGraph, share a single TypedDict state and execute sequentially per telemetry tick with conditional error routing and SQLite checkpointing.



All 10 Nodes

NODE	FILE	INPUT FROM STATE	OUTPUT TO STATE
n01_collect	Graph_node/n01_collect.py	last_processed_id	raw_metric, raw_log, raw_traces, episode_id, failure_mode, timestamp
n02_feature_eng	Graph_node/n02_feature_engineering.py	raw_metric, raw_log, raw_traces	classifier_input (37 features), evidence
n03_prelim_sev	Graph_node/n03_prelim_severity.py	classifier_input	preliminary_severity, severity_result
n04_classify	Graph_node/n04_classify.py	classifier_input	predicted_failure, prediction_probability
n05_tumbling_win	Graph_node/n05_tumbling_window.py	predicted_failure	dominant_state, vote_distribution, window_margin, window_full
n06_forecasting	Graph_node/n06_forecasting.py	dominant_state, classifier_input, episode_id	time_to_failure, forecast_confidence, threshold_crossed, forecast_algorithm
n07_severity_upd	Graph_node/n07_severity_update.py	preliminary_severity, forecast_result	revised_severity, impact_band, urgency_band, is_escalated, dwell_count
n08_reliability	Graph_node/n08_reliability.py	failure_mode, elapsed_s	survival_probability, weibull_beta, weibull_eta, active_failure_group
n09_human_gate	Graph_node/n09_human_gate.py	revised_severity, committed_severity	hg_needed, hg_review_id, hg_decision, hg_final_severity
n10_db_writer	Graph_node/n10_db_writer.py	full AIOpsLangState	Writes 11 tables; calls SSEBroadcaster.update()

Conditional Routing

AFTER N01_COLLECT

state.error == "no_data" → **END** (outer loop retries after POLL_INTERVAL_MS)
no error → **feature_eng**

AFTER N02-N09 (ANY NODE)

state.error is set → **db_writer** (write partial state, then END)
no error → next node in linear chain

Tumbling Window — Label Smoother

Raw per-cycle classifier output is inherently noisy. The tumbling window accumulates 10 consecutive predictions and selects the majority label, suppressing transient misclassifications.

The LightGBM classifier makes a statistically independent decision every 500ms. Because real metrics have ±4% jitter, a **MEMORY_LEAK** episode might produce an occasional **CPU_SATURATION** or **NONE** prediction during a single anomalous tick. Without smoothing, the pipeline would fire false severity escalations and TTF forecasts for failure modes that were only momentarily dominant.

Node **n05_tumbling_window** uses a `collections.deque(maxlen=10)` buffer. Each cycle, the new prediction is appended (the oldest is dropped when full). A majority vote determines the `dominant_state`.

Mathematical Definition

dominant_state = argmax_{mode} (count of mode in buffer)

margin = votes_{winner} - votes_{runner-up}

buffer = deque of last N=10 predicted_failure strings

votes_m = number of times mode m appears in buffer

margin = confidence indicator (high margin → stable dominant state)

Worked Example

CYCLE	CLASSIFIER OUTPUT	BUFFER CONTENTS	DOMINANT STATE	MARGIN	WINDOW FULL
1	MEMORY_LEAK	[ML]	MEMORY_LEAK	1	No
3	CPU_SATURATION	[ML, ML, CS]	MEMORY_LEAK	1	No
7	MEMORY_LEAK	[ML,ML,CS,ML,ML,ML,ML]	MEMORY_LEAK	5	No
10	MEMORY_LEAK	[ML,ML,CS,ML,ML,ML,ML,ML,CS,ML]	MEMORY_LEAK	6	Yes

At cycle 10: vote_distribution = {"MEMORY_LEAK": 8, "CPU_SATURATION": 2}, margin = 8-2 = 6.

Design Rules

WHAT IS SMOOTHED

Only the **label sequence** (Y) — the string of predicted failure mode names across 10 cycles. This is the classifier's categorical output.

WHAT IS NOT SMOOTHED

Raw metric features. Averaging 10 cycles of CPU% would destroy genuine spikes. A CPU spike at cycle 8 would be diluted by 7 calm cycles before it.

EPISODE BOUNDARY BEHAVIOR

The buffer is **not reset** when the `episode_id` changes. Labels carry forward across episode boundaries to prevent cold-start instability when a new episode begins. This means the window's dominant state reflects the recent historical trend, not just the current episode.

Output State Fields

FIELD	TYPE	DESCRIPTION
dominant_state	str	Majority-vote winner from the 10-cycle buffer (used by Forecasting)
vote_distribution	dict	JSON map of all modes → vote counts in current buffer
window_margin	float	Winner votes minus runner-up votes; higher = more stable classification
window_full	bool	True once the buffer has ≥ 10 predictions; gates forecasting decisions

Forecasting & Time-to-Failure

Each failure mode has a unique degradation trajectory. The forecasting engine routes the dominant failure state to a dedicated algorithm that computes the seconds remaining before a critical threshold breach.

Node **n06_forecasting** calls `route_forecast()`, which dispatches to one of 12 dedicated forecasting functions based on `dominant_state`. Each function maintains a per-episode rolling buffer of feature rows and fits its model to that buffer to compute the current slope or trajectory.

Algorithm Routing Table

FAILURE MODE	PRIMARY METRIC	ALGORITHM	THRESHOLD
MEMORY_LEAK	heap_mb	Linear Extrapolation	Max heap capacity
CPU_SATURATION	cpu_utilization	auto_arma / Exponential	95%
LATENCY_SPIKE	p99_latency	auto_arma	1,000 ms
DB_SLOWDOWN	db_p99	Linear Extrapolation	500 ms
CACHE_STAMPEDE	cache_miss_rate	Linear Extrapolation	80%
QUEUE_BACKUP	queue_lag	Linear Extrapolation	Critical lag
DEPENDENCY_TIMEOUT	upstream_timeout_rate	Threshold Convergence	50%
ERROR_STORM	error_rate	auto_arma	10%
RETRY_STORM	retry_count_per_request	Linear Extrapolation	Max retries
DISK_IO_SATURATION	iops_utilization	Linear Extrapolation	95%
BAD_DEPLOY	error_rate	Step-function Detection	Immediate
CASCADING_FAILURE	Multi-feature	Convergence (min TTF)	Any P1 threshold

Forecasting Algorithms — Formulas

1. Linear Extrapolation

$$TTF = (Threshold - Current) \div Slope$$

Slope = $\Delta value / \Delta time$ (regression over buffer)
Threshold = critical breach value for this mode
TTF = seconds until breach (None if slope ≤ 0)

2. Exponential Growth

$$TTF = \ln(Threshold / Current) \div r$$

r = exponential growth rate (fit to log of values)
 $\ln$ = natural logarithm
TTF = None if $Current \geq Threshold$ (already breached)

Used for: CPU_SATURATION in accelerating phase

Used for: MEMORY_LEAK, DB_SLOWDOWN, CACHE_STAMPEDE, QUEUE_BACKUP, RETRY_STORM, DISK_IO_SATURATION

3. ARIMA (Auto-Regressive Integrated Moving Average)

$$y_t = c + \sum \phi_i \cdot y_{t-i} + \sum \theta_j \cdot \epsilon_{t-j} + \epsilon_t$$

ϕ_i = autoregressive coefficients (past values)

θ_j = moving-average coefficients (past errors)

ϵ_t = white noise error term

`auto_arima` from `pmdarima` selects optimal (p,d,q) order automatically

TTF is computed by finding the first forecast horizon where predicted value $\geq$ threshold

4. Multi-Feature Convergence (CASCADING_FAILURE)

Each feature's individual TTF is computed via its own algorithm. The **minimum TTF across all features** is reported, along with `earliest_ttf_feature` (which metric will breach first):

$$TTF_{final} = \min(TTF_{cpu}, TTF_{mem}, TTF_{p99}, TTF_{error}, \dots)$$

`earliest_ttf_feature` = the feature name with minimum TTF

FORECAST CONFIDENCE GATE

Before a TTF estimate influences severity, it must pass a confidence gate. If `forecast_confidence` < 0.75 (`min_confidence`), the `urgency_band` defaults to "Distant" regardless of TTF value. This prevents premature severity escalation when the forecast model has insufficient history (buffer length < minimum required cycles).

Severity Update & Hysteresis

Preliminary severity answers "how bad is it now?" — but severity must also reflect urgency (how fast is it getting worse?) and stability (is this a genuine trend, or a transient spike?). The severity update node provides all three.

Node **n07_severity_update** implements a four-step process that cross-references the preliminary severity (current impact) with the forecast result (future urgency) to produce a **revised severity** that is then stabilized through hysteresis to prevent severity flapping.



Step 1: Impact Band Mapping

PRELIMINARY SEVERITY	IMPACT BAND
P1	High
P2	Moderate
P3	None
P4	None

Step 2: Urgency Band (Gated by Confidence)

```
URGENCY_BAND_LOGIC

if forecast_confidence < 0.75:
    urgency_band = "Distant"      # gate FAILS — not enough forecast history
elif time_to_failure < 60:
    urgency_band = "Imminent"    # < 60 seconds to breach
elif time_to_failure < 300:
    urgency_band = "Near"        # 60-300 seconds to breach
else:
    urgency_band = "Distant"      # > 300 seconds or no TTF
```

Step 3: Impact × Urgency Matrix

	IMMINENT (TTF < 60S)	NEAR (60-300S)	DISTANT (> 300S)
High	P1 — CRITICAL	P1 — CRITICAL	P2 — HIGH
Moderate	P1 — CRITICAL	P2 — HIGH	P3 — MEDIUM

	IMMINENT (TTF < 60S)	NEAR (60-300S)	DISTANT (> 300S)
None	P2 — HIGH	P3 — MEDIUM	P4 — LOW

Step 4: Hysteresis Reconciliation

Escalation: revised = candidate (immediate, if candidate > current)

De-escalation: revised = candidate only if dwell_count ≥ k

k = dwell_k = 5 cycles (default = 10 seconds of telemetry)
dwell_count: increments each cycle that candidate < current
Resets to 0 whenever candidate changes or matches current

THE CORE INSIGHT

Escalation is always immediate — a jump to P1 fires right away because the cost of delayed escalation is an unresolved critical failure. De-escalation is deliberately slow — a drop to P3 requires 5 consecutive cycles at the lower level before it is accepted. This prevents false "recovery" signals from a single clean tick after 9 bad ones.

Reliability & Survival Analysis

Beyond detecting that a failure is happening, Sentinel quantifies the historical probability of system survival — how long episodes of this type typically last before they terminate or resolve.

Node `n08_reliability` applies **Kaplan-Meier non-parametric estimation** and **2-Parameter Weibull Maximum Likelihood Estimation** to the historical episode database. Together, these tell the dashboard: "given that this service is in a `MEMORY_LEAK` episode that has been running for 90 seconds, what is the survival probability?"

Four Failure Mode Groups (Weibull Stratification)

GROUP	FAILURE MODES	EXPECTED B	BEHAVIOR
Immediate Trigger	BAD_DEPLOY, CACHE_STAMPEDE, CASCADING_FAILURE, CPU_SATURATION, DEPENDENCY_TIMEOUT, ERROR_STORM	> 1	Wear-out; failure rate increases over time
Fast Accumulation	QUEUE_BACKUP, RETRY_STORM	> 1	Rapid growth pattern, high early failure rate
Progressive Degradation	DISK_IO_SATURATION	≈ 1	Near-constant hazard rate
Slow / Latent Degradation	DB_SLOWDOWN, MEMORY_LEAK, LATENCY_SPIKE	< 1 → 1+	Slow onset; hazard increases late

Kaplan-Meier Non-Parametric Survival Estimate

The KM estimator computes a step-function survival curve from historical episode durations, handling right-censored episodes (episodes that ended before resolution):

$$S(t) = \prod_{t_i \leq t} (1 - d_i / n_i)$$

d_i = number of episodes that ended (failed) at time t_i
 n_i = number of episodes at risk just before t_i (not yet ended or censored)
 $S(0) = 1.0$ (all episodes survive at time zero)

Greenwood Standard Error — 95% confidence interval for the KM curve:

$$SE(t) = S(t) \times \sqrt{ \sum_{t_i \leq t} d_i / (n_i(n_i - d_i)) }$$

2-Parameter Weibull Maximum Likelihood Estimation

The Weibull distribution models failure time with two parameters: **β (shape/slope)** and **η (scale/characteristic life)**:

$$f(t; \beta, \eta) = (\beta/\eta) \cdot (t/\eta)^{(\beta-1)} \cdot \exp(-(t/\eta)^\beta)$$

$$R(t) = \exp(-(t/\eta)^\beta) \text{ [Survival / Reliability function]}$$

$$h(t) = (\beta/\eta) \cdot (t/\eta)^{(\beta-1)} \text{ [Hazard rate]}$$

$\beta < 1$: decreasing hazard rate (early failures; "burn-in" period)
 $\beta = 1$: constant hazard rate (random / exponential failure)
 $\beta > 1$: increasing hazard rate (wear-out; most production failure modes)
 η : scale parameter — time at which 63.2% of episodes have ended

MLE Fitting Priority

- 1 reliability library (Reid) — Primary**
 Most accurate for small samples; handles right-censored data with `Fit_Weibull_2P()`
- 2 lifelines (Davidson-Pilon) — Secondary**
`WeibullFitter().fit(); rho_ →  $\beta$ , lambda_ →  $\eta$`
- 3 SciPy L-BFGS-B — Fallback**
 Native negative log-likelihood minimization; always available

Runtime Survival Probability

In the live pipeline, pre-fitted Weibull parameters (`weibull_params.json`) are loaded once at startup. For each cycle, the current episode's `elapsed_s` is used to compute survival probability:

$$P(\text{survival at elapsed_s}) = \exp(-(elapsed_s / \eta)^\beta)$$

Written to state as `survival_probability`, displayed in the frontend `ReliabilityCard`.

Human-in-the-Loop Gate

Fully automated severity escalation is appropriate for gradual changes — but a sudden jump from P4 to P1 should not wake up an entire on-call team without a human confirming it first.

Node **n09_human_gate** intercepts the pipeline when a **large severity jump** is detected. The pipeline does not proceed to **n10_db_writer** until the operator either approves or rejects the proposed severity change — or until the auto-approval timeout fires.

Trigger Condition

```
ESCALATION_DETECTOR.PY - TRIGGER LOGIC

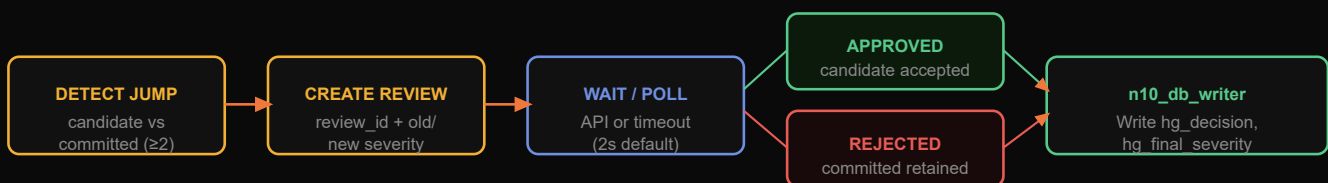
# Map severity to numeric level: P1=4, P2=3, P3=2, P4=1
committed_level = SEV_MAP[committed_severity] # last confirmed severity
candidate_level  = SEV_MAP[candidate_severity] # proposed new severity

jump = candidate_level - committed_level

# Large jump: ≥ 2 severity levels OR first-time P1
is_large_jump = (jump >= 2) or (candidate_severity == "P1" and
                                committed_severity not in ("P1",))

if is_large_jump:
    create_review(episode_id, committed_severity, candidate_severity)
    state["hg_needed"] = True
```

Gate Lifecycle



Auto-Approval Timeout

The gate waits up to **HUMAN_GATE_TIMEOUT_SECONDS = 2** seconds (configurable in `config.py`) for an operator decision. If no decision arrives in that window, the `timeout_manager` auto-approves with decision **AUTO_APPROVED** and operator "auto-timeout". This prevents the pipeline from stalling indefinitely in production.

Audit Trail — node_human_gate Table

FIELD	DESCRIPTION	EXAMPLE
review_id	UUID primary key for this review	a3f2-91bc-...
old_severity	Committed severity before the jump	P4
new_severity	Proposed (candidate) severity	P2
final_severity	Actual severity applied after decision	P2
decision	APPROVED / REJECTED / AUTO_APPROVED	APPROVED
operator	Username of reviewer (or "auto-timeout")	kunal.w
response_ms	Milliseconds from create to decision	1,840
escalation_summary	Human-readable escalation context	—

DESIGN PRINCIPLE

The Human Gate does **not** block the pipeline thread. It uses an interrupt mechanism (`interrupt_manager.py`) that polls for a decision while the background simulator and other processes continue running. Only the final DB write for this specific episode's severity is deferred until the gate resolves.

QueryLangGraph — Chatbot Agent

Instead of writing SQL, operators ask: "What caused the P1 incident at 3am?" — and a guarded LLM agent pipeline retrieves the data and synthesizes a human-readable answer.

The **QueryLangGraph** chatbot is a separate FastAPI service (port 8001) powered by its own 11-node StateGraph. It is entirely independent of the inference pipeline — it reads from the same SQLite databases but does not write to them. Its architecture is designed around a principle of **defense in depth**: four separate guardrail nodes screen the query at different stages before any LLM inference is triggered.

11-Node Guarded Pipeline

#	NODE	TYPE	PURPOSE	GATE CONDITION
1	parse_guard	GUARDRAIL	Block SQL injection, prompt injection, PII queries	is_safe = False → respond
2	parse_query	PROCESSING	NLP extraction: intent + entities from query	—
3	validation_guard	GUARDRAIL	Validate parsed output format against expected schema	is_safe = False → respond
4	validation	PROCESSING	Semantic validation: entities against DB schema	is_validated = False → respond
5	intent_router	ROUTER	Route to correct query strategy (episode_lookup, trend, etc.)	—
6	retrieval_guard	GUARDRAIL	Prevent unauthorized DB access patterns	is_safe = False → respond
7	retrieval	PROCESSING	Execute structured SQLite query → raw data	—
8	sufficiency_router	ROUTER	Check if retrieved data is sufficient to answer	is_sufficient = False → respond
9	synthesis_guard	GUARDRAIL	Validate raw data before passing to LLM	is_safe = False → respond
10	synthesis	LLM	Google Gemini / OpenAI generates human-readable response	—
11	respond	OUTPUT	Format final JSON response payload	→ END

QueryState Schema

Example Queries

```
class QueryState(TypedDict):
    user_query: str
    parsed_intent: dict
    is_safe: bool
    is_validated: bool
    is_sufficient: bool
    retrieved_data: dict
    synthesized_response: str
    final_response: dict
```

- "What was the root cause of the last P1 incident?"
- "Show me all MEMORY_LEAK episodes in the last 24 hours"
- "What is the time-to-failure for the current episode?"
- "How many Human Gate reviews were auto-approved today?"

Fallback — PurePythonQueryGraphRunner

If the `langgraph` package is not installed, a `PurePythonQueryGraphRunner` executes the same 11 nodes sequentially without the `StateGraph` routing infrastructure. This guarantees that the chatbot remains functional even in minimal environments.

IMPORTANT: SEPARATION OF CONCERNS

The `QueryLangGraph` is a **read-only agent**. It queries existing incident data from SQLite but never writes to it. The LLM synthesis step uses Google Gemini (or OpenAI) only to convert retrieved structured data into natural language — it cannot modify incidents, change severities, or trigger pipeline actions.

API & SSE Architecture

A FastAPI service on port 8080 serves all REST endpoints and the real-time SSE stream that drives the React dashboard. A second FastAPI service on port 8001 handles chatbot queries.

Complete API Reference — Port 8080

METHOD + PATH	PURPOSE	RESPONSE
GET /api/health	Service health check	{"status": "healthy"}
GET /api/live	Latest pipeline state (historical CSV)	Full incident object
GET /api/episodes	All historical episodes list	List[EpisodeSummary]
GET /api/episodes/{id}	Full detail for one historical episode	EpisodeDetail
PATCH /api/incident/{id}	Update incident status (OPEN/ACK/IN_PROGRESS/RESOLVED)	StatusResponse
GET /api/reliability/summary	4-group Weibull params + KM curves	ReliabilitySummary
GET /api/live-stream	SSE stream — real-time pipeline state every 500ms	text/event-stream
GET /api/live-feed/state	Latest live pipeline state (REST fallback for SSE)	LiveFeedState
GET /api/live-feed/episodes	All episodes from live_feed_db.sqlite	List[Episode]
GET /api/live-feed/status	Queue depth, episode count, latest mode	StatusObject
POST /api/live/start	Start simulator + LangGraph subprocesses	{"status": "started"}
POST /api/live/stop	Terminate both subprocesses	{"status": "stopped"}
GET /api/live/simulation-status	Check if subprocesses are alive	{"running": true, ...}
GET /api/human-gate/pending	All reviews awaiting operator decision	List[ReviewSummary]
GET /api/human-gate/review/{id}	Full review detail + marks as REVIEWING	ReviewDetail
POST /api/human-gate/decision/{id}	Submit APPROVED / REJECTED decision	DecisionResult
GET /api/human-gate/metrics	Gate KPIs: approval rate, avg response time	GateMetrics
GET /api/human-gate/history	Recent audit records, newest first	List[AuditRecord]

SSE Broadcaster Architecture

```

class SSEBroadcaster:
    _lock: threading.Lock # protects _latest_state
    _latest_state: dict # most recent pipeline state

    @classmethod
    def update(cls, state: dict): # called from pipeline thread (sync)
        with cls._lock:
            cls._latest_state = safe_serialize(state)

    @classmethod
    async def stream(cls, interval_s=0.5): # async generator for FastAPI
        while True:
            payload = LiveFeedService.get_live_feed_state()
            yield f"data: {json.dumps(payload)}\n\n"
            await asyncio.sleep(interval_s)

```

The pipeline thread (synchronous) calls `SSEBroadcaster.update()` after every `n10_db_writer` cycle. The FastAPI SSE endpoint uses an async generator that yields the latest enriched dashboard payload every 500ms to all connected clients simultaneously. Thread safety is guaranteed by the `threading.Lock`.

PROCESS MANAGEMENT VIA API

`POST /api/live/start` spawns two subprocesses stored as module-level globals `_live_feed_proc` and `_langgraph_proc`. Liveness is checked with `proc.poll()` is `None`. Termination uses `proc.terminate()` which sends `SIGTERM`, allowing graceful shutdown. The subprocess approach avoids threading complexity while giving the API full lifecycle control.

Frontend Dashboard

A React + Vite single-page application subscribes to the SSE stream and renders live incident state — severity, failure mode, forecast, survival probability, and chatbot — all updated every 500 milliseconds.

13 UI Components

TopHeader

TopHeader.jsx

App title + Live Feed on/off toggle button

→ POST /api/live/start · /api/live/stop

Sidebar

Sidebar.jsx

Historical episode list navigation panel

→ GET /api/episodes

SummaryCard

SummaryCard.jsx

Current incident headline: mode, severity, status

→ SSE incident state

PredictionCard

PredictionCard.jsx

LightGBM output: predicted_failure + confidence score

→ SSE predicted_failure, prediction_probability

DiagnosisCard

DiagnosisCard.jsx

Root-cause diagnosis and evidence display panel

→ SSE evidence, dominant_state

EvidenceCard

EvidenceCard.jsx

Raw telemetry evidence: anomalous metrics listing

→ SSE raw_metric, raw_log

MultiTrendCharts

MultiTrendCharts.jsx

7 live time-series metric charts via Recharts library

→ SSE chart_data

AIOpsChartRenderer

AIOpsChartRenderer.jsx

Dynamic chart selector for individual metric view

→ MultiTrendCharts sub-component

ReliabilityCard

ReliabilityCard.jsx

Weibull survival curve + Kaplan-Meier overlay chart

→ SSE reliability fields

ReliabilityGraphSection

ReliabilityGraphSection.jsx

Full 4-group reliability dashboard with survival curves

→ GET /api/reliability/summary

HumanGatePanel

HumanGatePanel.jsx

Review queue with APPROVE / REJECT action buttons

→ GET /api/human-gate/pending

AIOpsChat

AIOpsChat.jsx

LLM chatbot panel — queries QueryLangGraph agent

→ POST http://localhost:8001/query

□ POST / Action endpoint □ SSE real-time stream □ GET / REST polling

SSE Connection Lifecycle

APP.JSX — EVENTSOURCE SETUP

```
// Established on component mount; auto-reconnects on disconnect
const es = new EventSource('http://localhost:8080/api/live-stream');

es.onmessage = (e) => {
  const data = JSON.parse(e.data);
  setIncident(data);      // triggers component re-renders
};

es.onerror = () => {
  // Browser auto-reconnects after a brief delay
};
```

STATE MANAGEMENT PATTERN

App.jsx owns all shared state via React `useState` hooks. There is no Redux, Zustand, or Context API — the SSE payload is rich enough that each component receives its needed fields directly as props. The SSE payload is enriched server-side by `LiveFeedService.get_live_feed_state()` before transmission, so the frontend performs zero data transformation.

Database & Persistence

All telemetry, pipeline outputs, and audit records are persisted to SQLite using WAL mode for concurrent read/write access across multiple processes.

Two SQLite Databases

DATABASE	PATH	WRITTEN BY	READ BY
simulator_db.sqlite	Simulator/app_data_generator_for_offline/output/	Offline simulator	run_langgraph.py (historical)
live_feed_db.sqlite	Simulator/live_feed_simulator/output/	run_live_feed.py	run_langgraph.py --live, FastAPI services

WAL Mode — Why It Matters

SQLite's default journal mode locks the database file during writes, which would block the LangGraph reader while the simulator is writing. **WAL (Write-Ahead Logging)** mode appends writes to a separate WAL file without blocking readers. Readers see the last committed checkpoint; the writer never locks the main database file.

```
DBWRITER.SETUP() - WAL CONFIGURATION

conn.execute("PRAGMA journal_mode=WAL")      # concurrent reads OK
conn.execute("PRAGMA synchronous=NORMAL")    # faster, still crash-safe
conn.execute("PRAGMA foreign_keys=ON")       # referential integrity
```

All 11 Tables

TABLE	WRITTEN BY	KEY COLUMNS
metrics	Simulator	id (PK, AUTOINCREMENT), episode_id, timestamp, 32 metric columns
logs	Simulator	episode_id, timestamp, log_level, exception_type, log_message
traces	Simulator	episode_id, timestamp, trace_id, span_id, span_duration_ms, span_status
node_feature_engineering	n10_db_writer	cycle, episode_id, 37 feature columns
node_preliminary_severity	n10_db_writer	preliminary_severity, weighted_score, blast_size, reason
node_classification	n10_db_writer	predicted_failure, prediction_probability
node_tumbling_window	n10_db_writer	dominant_state, vote_distribution (JSON), window_margin, window_full
node_forecasting	n10_db_writer	time_to_failure, forecast_confidence, algorithm_used, feature_ttf (JSON)
node_severity_update	n10_db_writer	impact_band, urgency_band, revised_severity, is_escalated, dwell_count
node_reliability	n10_db_writer	group_name, survival_probability, beta, eta, ttf_seconds
node_human_gate	n10_db_writer	review_id (PK), decision, operator, response_ms, is_large_jump
pipeline_results	n10_db_writer	Denormalized: all node outputs combined (50+ columns) — primary read by API

THE CORE INSIGHT — DBWRITER DESIGN

The `DbWriter` class in `nodes/db_writer/db_writer.py` uses Python's `with conn:` context manager for atomic transactions. Every node's output is written as a single `INSERT` OR `IGNORE` statement within one transaction. If any write fails, the entire transaction rolls back — no partial row is ever committed to the database.

Technology Stack

Every technology in Sentinel was chosen for a specific engineering reason — not default convention — and each serves a clearly bounded responsibility.

Backend Stack

TECHNOLOGY	VERSION	ROLE	WHY THIS CHOICE
Python	3.11+	Language	Mature ML/data ecosystem; type hints for StateGraph correctness
LangGraph	0.1.x	Pipeline Orchestration	Stateful directed graph with conditional routing + SQLite checkpointing; native StateGraph primitive
LightGBM	4.x	ML Classifier	Leaf-wise growth faster than XGBoost on 37 features; native predict_proba(); no normalization needed
Optuna	3.x	HPO	TPE sampler outperforms grid/random search for 9-parameter LightGBM space
Drain3	0.9.x	Log Parser	Streaming log template miner; detects novel patterns without retraining
FastAPI	0.111.x	REST + SSE API	Native async + StreamingResponse for SSE; automatic OpenAPI docs
Uvicorn	0.29.x	ASGI Server	High-performance ASGI runtime for FastAPI async endpoints
SQLite (WAL)	3.x	Persistence	Embedded; WAL mode enables concurrent reads from multiple processes without locking
Pandas	2.x	Data Processing	Feature engineering, CSV I/O, episode aggregation
SciPy	1.13.x	Statistical Analysis	L-BFGS-B optimization for Weibull MLE fallback
pmdarima	2.x	Time-Series Forecast	auto_arima automatically selects ARIMA (p,d,q) order; no manual tuning
Pydantic v2	2.x	State Validation	Field-level validation of AIOpsLangState at every node boundary
Google Gemini / OpenAI	latest	LLM Synthesis	QueryLangGraph synthesis node; natural language generation from structured DB results

Frontend Stack

TECHNOLOGY	VERSION	ROLE	WHY THIS CHOICE
React	18.x	UI Framework	Component-based; fine-grained re-rendering on SSE state updates
Vite	5.x	Build Tool	Sub-second HMR; ES module native bundling; faster than Create React App
Recharts	2.x	Charting	React-native charts; AreaChart and LineChart for time-series; ComposedChart for KM overlay
Vanilla CSS	—	Styling	Full design-system control; no Tailwind configuration overhead; custom dark theme
EventSource (browser)	native	SSE Client	Native browser API; auto-reconnect; no additional library needed

The AIOps State Schema

The `AIOpsLangState TypedDict` is the single shared memory that flows through all 10 pipeline nodes. Every node reads from it and returns only the keys it updated — `LangGraph` merges the partial dict into the full state.

Cycle Identity

FIELD	TYPE
cycle	int — pipeline cycle counter
last_processed_id	int — last SQLite metrics row id consumed

Raw Telemetry

FIELD	TYPE
raw_metric	dict — 35 columns from metrics table
raw_log	list[dict] — log entries for this tick
raw_traces	list[dict] — span entries for this tick
episode_id	str — ep_MEMORY_LEAK_20260801_001
failure_mode	str — ground truth from simulator
timestamp	float — Unix epoch
elapsed_s	float — seconds since episode start
service	str — auth-service, payment-service, etc.

Feature Engineering

FIELD	TYPE
classifier_input	dict — 37-feature vector for LightGBM
evidence	dict — anomalous metrics + diagnostic metadata

Classification

Tumbling Window

FIELD	TYPE
dominant_state	str — majority-vote label
vote_distribution	dict — {mode: count}
window_margin	float — winner – runner-up votes
window_full	bool — True after 10 predictions

Forecasting

FIELD	TYPE
forecast_result	dict — full raw forecasting output
forecast_algorithm	Optional[str] — algorithm used
time_to_failure	Optional[float] — seconds until breach
forecast_confidence	Optional[float] — 0.0–1.0
threshold_crossed	Optional[bool]
earliest_ttf_feature	Optional[str] — first-to-breach metric

Severity Update

FIELD	TYPE
preliminary_severity	str — P1 / P2 / P3 / P4
severity_result	dict — weighted score, blast_size, reason
predicted_failure	str — MEMORY_LEAK, CPU_SATURATION, etc.
prediction_probability	float — confidence 0.0–1.0

FIELD	TYPE
revised_severity	Optional[str] — final P1–P4
impact_band	Optional[str] — High / Moderate / None
urgency_band	Optional[str] — Imminent / Near / Distant
is_escalated	Optional[bool]
is_deescalated	Optional[bool]
dwelling_count	Optional[int]

Human Gate

FIELD	TYPE
hg_needed	Optional[bool]
hg_review_id	Optional[str] — UUID
hg_decision	Optional[str] — APPROVED / REJECTED / AUTO
hg_final_severity	Optional[str]
hg_response_ms	Optional[int]
error	Optional[str] — triggers db_writer shortcut

TECHNICAL NOTE — TYPEDDICT + PYDANTIC

LangGraph's StateGraph requires a TypedDict (not a Pydantic BaseModel). A mirrored _AIOpsStateModel (Pydantic BaseModel) is used only for validation at node boundaries via the validate_state() helper. This provides field-level type coercion and range checks without changing the underlying dict-based state mechanism.

End-to-End Data Flow

A single telemetry tick travels through seven distinct layers in under 500 milliseconds — from raw metric generation to a rendered React dashboard update.

1 Simulator generates one tick (0.5s interval)

`run_live_feed.py` writes one row each to metrics, logs, and traces tables in `live_feed_db.sqlite` using WAL-mode INSERT. Each row is identified by (episode_id, timestamp).

2 n01_collect reads the new row

`run_langgraph.py` polls the DB every 100–500ms. When a new row appears with `id > last_processed_id`, n01 fetches the metric row plus correlated logs and traces by (episode_id, timestamp), populating `AI0psLangState`.

3 n02 builds the 37-feature vector

32 metric fields are extracted from `raw_metric` with type coercion. 5 log features are computed from `raw_log` (count, severity, critical count, exception flag, Drain3 novel template). Result written to `state.classifier_input`.

4 n03 assigns preliminary severity P1–P4 (microseconds)

Rule engine evaluates threshold conditions in order `P1 → P2 → P3 → P4`. The first triggered level is assigned. A human-readable reason string (e.g., "cpu=91.2%>90%") is written to the state.

5 n04 classifies the failure mode (LightGBM inference, <1ms)

Single-row `DataFrame` → `model.predict_proba()` → `argmax label` → `inverse_transform()`. Sets `state.predicted_failure` and `state.prediction_probability`.

6 n05 majority-votes the prediction into dominant_state

New prediction appended to `deque(maxlen=10)`. Counter applied. Winner and margin computed. `state.dominant_state` reflects the stable label; `state.window_full` gates forecasting.

7 n06 computes time-to-failure (mode-specific algorithm)

`router.route_forecast(dominant_state, episode_id, features)` dispatches to the dedicated forecasting function. Episode buffer receives the new feature row. Linear/ARIMA/exponential algorithm computes TTF. `state.time_to_failure` set in seconds.

8 n07 derives revised severity through impact × urgency matrix + hysteresis

`preliminary_severity` → `impact_band`. `TTF + confidence` → `urgency_band`. Matrix lookup → `candidate_severity`. Dwell counter determines whether to apply or hold the new severity.

9 n08 computes survival probability from Weibull parameters

Pre-fitted β and η for the current `failure_mode`'s group are loaded from `weibull_params.json`. $R(t) = \exp(-(elapsed_s / \eta)^\beta)$ is computed. `state.survival_probability` is set (0–100%).

10 n09 checks for large severity jump → Human Gate or pass-through

If candidate severity differs by ≥ 2 levels from `committed_severity`, a review is created and the gate waits up to 2 seconds for operator decision (or auto-approves).

11 n10 writes 11 tables + broadcasts SSE event

`DbWriter.write_pipeline_results()` writes the denormalized combined row to `pipeline_results`. All per-node tables are also updated. `SSEBroadcaster.update(state)` is called; the FastAPI `/api/live-stream` generator picks it up within 500ms and pushes it to connected browsers.

React re-renders all affected components

App.jsx receives the SSE event payload, calls `setIncident(data)`, triggering re-renders of `SummaryCard`, `PredictionCard`, `MultiTrendCharts`, `ReliabilityCard`, `HumanGatePanel` — all with fresh data from this cycle.

Outputs & Running the Project

Sentinel produces both structured database records and live dashboard visualizations — every pipeline output is persisted and observable.

Pipeline Output Files

FILE	LOCATION	CONTENT
engineered_features.csv	nodes/feature_engineering/output/	37-feature vector per cycle — primary training data for LightGBM
preliminary_severity.csv	nodes/preliminary_severity/output/	P1–P4 per cycle with weighted score, blast_size, and reason
pipeline_results.csv	nodes/classification/output/	Full classification results per cycle
tumbling_window_output.csv	nodes/tumbling_window/output/	Dominant state, vote distribution, margin per cycle
forecasting_output.csv	nodes/forecasting/output/	TTF predictions, algorithm used, confidence per cycle
severity_update_output.csv	nodes/severity_update/output/	Revised severity history with hysteresis dwell count
weibull_params.json	nodes/reliability/	Pre-fitted Weibull β and η per failure group
lgbm_model.pkl	nodes/classification/models/	Trained LightGBM classifier artifact
langgraph_checkpoints.db	nodes/classification/models/	LangGraph SqliteSaver — full state per node for crash recovery

Running the Project

STEP 1 – INSTALL DEPENDENCIES

```
pip install -r backend/requirements.txt
cd frontend && npm install
```

STEP 2 – TERMINAL 1: START ALL BACKEND SERVICES

```
cd d:/AIops_Incident_Management
python backend/run_server.py
# → Port 8001: QueryLangGraph Chatbot API (subprocess)
# → Port 8080: Sentinel Dashboard API (blocking)
```

STEP 3 – TERMINAL 2: START FRONTEND

```
cd d:/AIOps_Incident_Management/frontend
npm run dev
# → http://localhost:5173
```

STEP 4 – OPTIONAL: MANUAL PIPELINE

```
# Historical mode (offline data)
python backend/run_langgraph.py

# Live mode only (simulator + pipeline separate)
python backend/Simulator/live_feed_simulator/run_live_feed.py
python backend/run_langgraph.py --live

# Reset all databases and output files
python backend/reset_all.py
```

LIVE FEED — SINGLE-CLICK START

The simplest way to start the full live pipeline is to open <http://localhost:5173> and click the **"Start Live Feed"** button. This sends POST `/api/live/start`, which spawns both subprocesses and starts streaming telemetry within 2 seconds.

System Metrics at a Glance

Key numerical facts about the Sentinel AIOps implementation — all verified from actual source code.

13

FAILURE MODES

10

INFERENCE NODES

11

QUERY NODES

37

ML FEATURES

4

GUARDRAIL NODES

11

DATABASE TABLES

20+

API ENDPOINTS

3

SERVER PORTS

80

OPTUNA TRIALS

10

WINDOW SIZE

5

DWELL CYCLES K

2s

GATE TIMEOUT

Formulas Reference Card

MODULE	FORMULA
P1 Trigger	<code>cpu > 90 OR mem > 90 OR p99 > 1000 OR err > 0.10 OR crit_logs > 5</code>
Tumbling Window Margin	<code>margin = votes_winner - votes_runner_up</code>
Linear TTF	<code>TTF = (threshold - current) / slope</code>
Exponential TTF	<code>TTF = ln(threshold / current) / growth_rate</code>
ARIMA	$y_t = c + \sum \phi_i y_{t-i} + \sum \theta_j \epsilon_{t-j} + \epsilon_t$
Cascading TTF	<code>TTF_final = min(TTF_cpu, TTF_mem, TTF_p99, ...)</code>
Kaplan-Meier	$S(t) = \prod (1 - d_i/n_i)$ for all $t_i \leq t$
Weibull Survival	$R(t) = \exp(-(t/\eta)^\beta)$
Weibull Hazard	$h(t) = (\beta/\eta) \cdot (t/\eta)^{(\beta-1)}$
Hysteresis Dwell	<code>revised = candidate iff dwell_count ≥ k (de-escalation)</code>
Severity Matrix P1	<code>Impact=High n Urgency=Imminent → P1 Impact=High n Near → P1</code>

Port Reference

PORT 8080

Sentinel Dashboard API

REST + SSE. All incident endpoints, live feed control, human gate, reliability summary.

PORT 8001

QueryLangGraph Chatbot

Natural language query API. POST /query → 11-node pipeline → JSON response.

PORT 5173

React Dashboard

Vite dev server. SPA with 13 components, SSE subscriber, chatbot panel.

Conclusion

Sentinel AIOps demonstrates that the knowledge required for expert-level incident management can be encoded, structured, and executed autonomously — making institutional knowledge permanent and scalable.

Production infrastructure generates more observability data than any human can process in real time. Sentinel's architecture accepts that premise and builds around it: **the system correlates, the system classifies, the system forecasts, the system escalates** — and then it tells the human what it found and why.

Every design decision in Sentinel has an engineering reason. LangGraph was chosen because its StateGraph allows each analysis node to have a single, testable responsibility and a clear interface (the shared state dict). LightGBM was chosen because gradient boosted trees make no assumptions about feature normalization and produce calibrated probability outputs that gate downstream decisions. WAL-mode SQLite was chosen because it allows concurrent access from multiple processes without a message broker.

THE ARCHITECTURE'S GOLDEN RULES

1. **Telemetry is correlated before intelligence is applied.** Metrics, logs, and traces are joined by (episode_id, timestamp) before any feature is extracted.
2. **Each pipeline node has exactly one responsibility.** Collection, engineering, severity, classification, smoothing, forecasting, and persistence are distinct nodes — not mixed concerns.
3. **Rule-based and ML classification are complementary.** Preliminary severity (P1–P4 rules) answers "how bad is it now"; LightGBM answers "what kind of failure is this" — both are needed.
4. **Label stability is the responsibility of the tumbling window.** Raw per-cycle classifier output is inherently noisy; the 10-cycle majority vote provides the stable label that forecasting and severity update rely on.
5. **Forecasting contributes urgency, not severity.** Time-to-failure estimates gate the urgency_band which feeds into the severity matrix — forecasting influences severity but does not set it directly.
6. **Escalation is immediate; de-escalation is earned.** A severity jump fires instantly. A severity drop requires 5 consecutive confirming cycles (the hysteresis dwell counter).
7. **Human Gate preserves operator accountability.** Large severity jumps require explicit human confirmation or auto-timeout; every decision is logged to the audit table.
8. **The database is the source of truth, not in-memory state.** Every pipeline cycle writes to SQLite before the SSE broadcast; the API reads from the database, not from in-process variables.
9. **The chatbot is a read-only intelligence layer.** QueryLangGraph queries incident data and synthesizes language — it never modifies state, triggers pipeline actions, or escalates incidents.
10. **Graceful degradation is designed, not accidental.** Missing model → auto-train. No data → retry after poll interval. Node failure → db_writer writes partial state → checkpoint resume.

